

ResumaXing

SE 3354
Spring 2026
Srimathi Srinivasan

TEAM 7

Deepshikha Machireddy

Hanna Kellenberger

Josias Acha

Ivan Leong

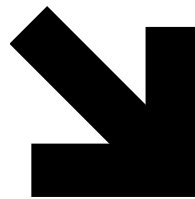


TABLE OF CONTENTS

Introduction	3
1. Requirements.....	4
2. Architecture Design.....	7
3. System Modeling	12
4. Wire Frames	17
5. Development Research & Plan	18
6. Implementation	21
7. Software Testing	22
8. Software Evolution	31
Glossary of Terms.....	32



INTRODUCTION

Purpose:

The purpose of the document is to provide the following information: the requirements discussed, the architectural design of the project, system modeling, wire frames, project implementation, software testing, and software evolution. It is further noted that the secondary purpose of the document is to provide the reader with a clear understanding of how Team 7 goes through the software engineering process to build a project of their choice.

Audience:

The intended audience of this document pertains to three groups of people. First and foremost, are the professor and the TAs. They are the main audience that will be judging our processes and systems. The most fundamental feedback will stem from their criticism and insights. The second intended audience are the potential students that may read upon this documentation. These can be guided towards those students within the same class section or those of future sections. Main reason for their inclusion is to provide descriptive yet simple documentation that demonstrates Team 7's project. The final group are individuals who gather interest in the project. They will need documentation to understand the process behind the project.

Structure:

The overall structure of the document is stated within the table of contents as well as the purpose above. The requirements section will discuss the functional and non-functional requirements as well as the root use cases. Architecture design will show the system architecture views (logical and physical) and the style that fits the project.[Insert system modeling] [Insert wire frames] [Insert implementation] [Insert software testing] [Insert software evolution] The glossary of terms will distinguish what important terminology will be used throughout the document.



1. REQUIREMENTS

Approach to gathering requirements:

Each member of the team came up with 2 use cases. Each member then created 4 functional requirements and 2-3 non-functional requirements for their respective use cases.

Use case approach – determine what systems there are and decide how the user interacts with said systems.

Functional requirement approach – break down the use case and find detailed requirements that stem from them

Non-functional requirement approach – take each functional requirement and create/brainstorm performance, security, and technical constraints.

The potential risks section of the SOW has been updated.

Use Cases:

- UC1: User should create a profile
 - FR-1.1: The user should be able to create user accounts
 - FR-1.2: The user should be able to create a username for the account
 - FR-1.3: The user should be able to create a password for the account
 - FR-1.4: The user should be able to input personal details
 - NFR-1.5: Account creation should be less than 3 seconds
- UC2: User log in to existing account
 - FR-2.1 The user should be able to log into their user account
 - FR-2.2: The user should be able to input their username onto the login page
 - FR-2.3: The user should be able to input their password onto the login page
 - FR-2.4: The user should be able to have a forget password option to reset the password
 - NFR-2.5: Database backs up every week
 - NFR-2.6: Systems shall prevent common attacks and secure user data
 - NFR-2.7: Systems should have 99% uptime
 - NFR-2.8: System should be able to support at least 100 users at the same time
 - NFR-2.9: System should recover from failure within 5 minutes
- UC3: User should be able to access resume history
 - FR-3.1: The user should be able to open the profile library tab
 - FR-3.2: The system should display the user's library
 - FR-3.3: The system should allow the user to choose from previous work
 - FR-3.4: The system should display the user's choice



- NFR-3.5: User friendly
- UC4: User should be able to select template resumes
 - FR-4.1: The user should be able to open the template tab
 - FR-4.2: The system should display the template page to user
 - FR-4.3: The system should allow the user to select a desired template
 - FR-4.4: The system should display the chosen template on the main page
- UC5: User should be able to view and use context menu
 - FR-5.1: The user should be able to download resume
 - FR-5.2: The user should be able to share resume
 - FR-5.3: The user should be able rename resumes
 - FR-5.4: The user should be able favorite resumes
 - FR-5.5: The user should be able to view context menu
 - NFR-5.5: Download should take 10 seconds
- UC6: User should be able to download the resume
 - FR-6.1: The user should be able to download resume as a .pdf
 - FR-6.2: The user should be able to download resume as a .docx
 - FR-6.3: The user should be able to download resume as a .txt
 - FR-6.4: The user should be able to download resume as a .md
 - NFR-6.5: Template generation should be less than 5 seconds
- UC7: User should be able to enter existing job and skills
 - FR-7.1: The system should allow users to enter past job experience
 - FR-7.2: The system should allow users to enter education
 - FR-7.3: The system should allow users to enter skills
 - FR-7.4: The system should allow users to enter professional summary
 - NFR-7.5: Page load time should be less than 5 seconds
- UC8: User can approve recommendations for resume
 - FR-8.1: The system should allow the user to enter job description
 - FR-8.2: The system should show users most used words in the description
 - FR-8.3: The user should be able to accept or decline system recommendations provided by the system
 - FR-8.4: The user should be able to add accepted recommendations to resume
- UC9: User should be able to favorite resume
 - FR-9.1: The user should be able to remove resume from favorites
 - FR-9.2: The user should be able to sort favorite resumes
- UC10: User should be able to share resume
 - FR-10.1: The user should be able to share resumes through a link
 - FR-10.2: The user should be able to embed resumes
 - FR-10.3: The user should be able to share resume via email
- UC11: User should be able to change font of resume



- FR-11.1: The user should be able to increase the font size
 - FR-11.2: The user should be able to decrease the font size
 - FR-11.3: The user should be able to change the font type
 - FR-11.4: The user should be able to change the font color
 - FR-11.5: The user should be able to change the font format (bold, underline, italicize)
- UC12: User should be able to change the name of the resume
- UC13: User should be able to change the theme of resume
 - FR-13.1: The user should be able to horizontally align
 - FR-13.2: The user should be able to vertically align
- UC14: User should be able to reorder sections of the resume
 - FR-14.1: The user should be able to move sections
- UC15: User should be able to add a new section to the resume
 - FR-15.1: The user should be able to add a new section
 - FR-15.2: The user should be able to remove a section



2. ARCHITECTURE DESIGN

Context Diagram:

The context diagram determines what goes into the Resumaxing main system. As seen below, the diagram pictures a messaging API, template API, user, and database. The messaging API is for the overall user profile and login process. It is needed to send messages to the users regarding login/signup information. The template API is needed to retrieve resume templates for the users to use. The user in the main system does the following: login, select template, work on template, download finished resume. The database stores all the necessary information from the user: previous work, user data, and resumes

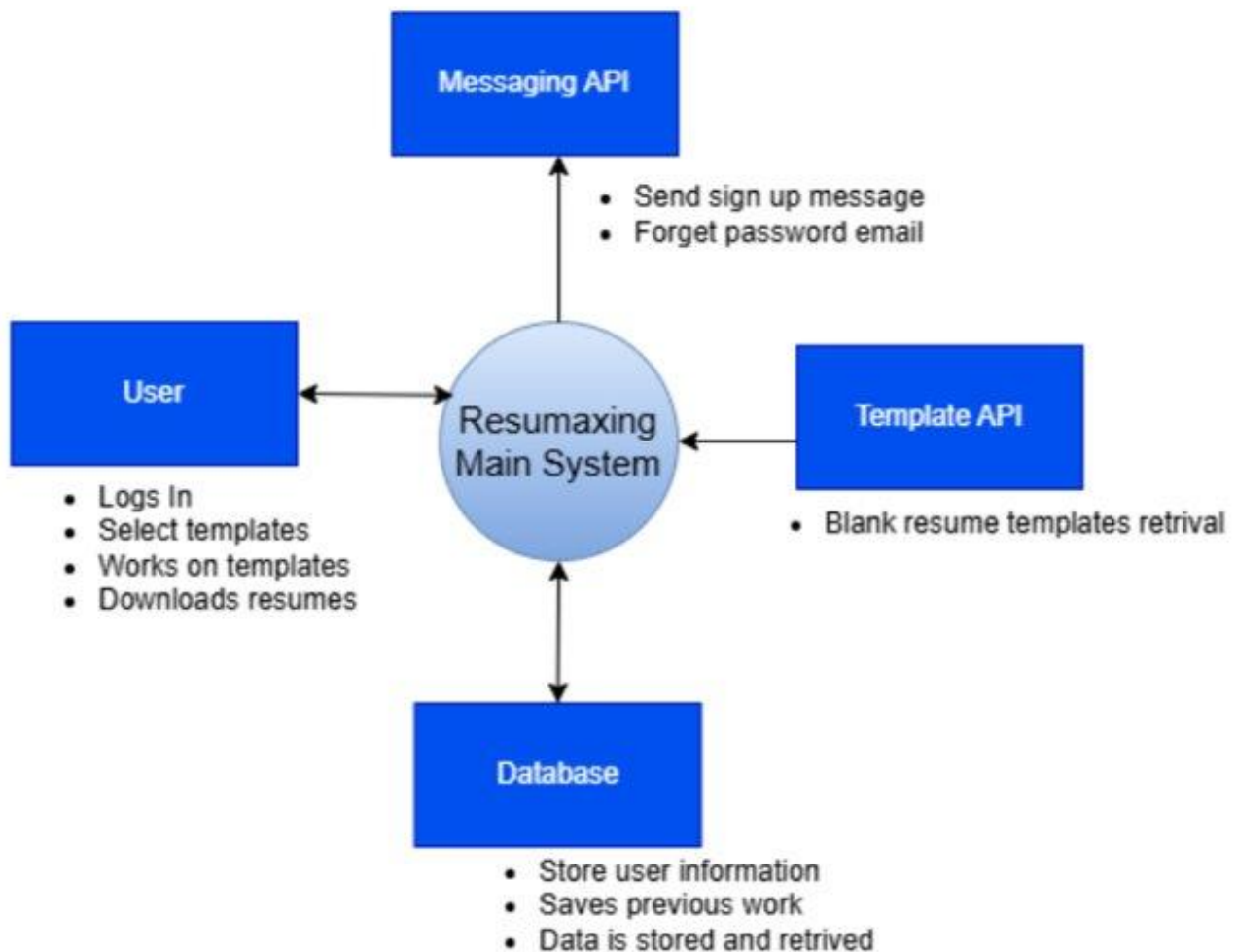


Fig. 1.1. Context Diagram



Logical View:

The logical view contains the user interface (UI) /front-end layer, back-end layer, database layer, and communication layer (API layer). The general idea is to allocate different items, tasks, and objects to each layer to define the structure of that layer. Within the user layer, contains user authentication, profile management, dashboard, and an editor mode. The main idea of this is to have a UI that allows users to freely navigate the project and use it with ease.

The back-end layer is where all the behind-the-scenes processes occur. This is where we have services for the user and resume. The main functionality lies behind this layer. Within the database layer is where all the objects that need to be retained are stored. This includes the user's profile, their resumes, templates, and data. The API layer allows for overall communication. It determines what APIs need to be used for the overall project to work.

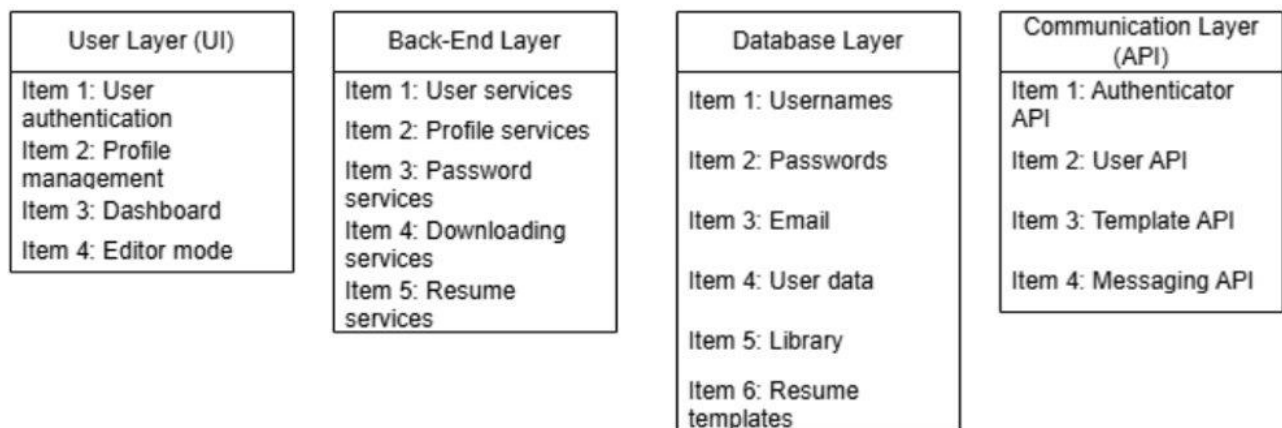


Fig. 1.2. Logical View

Physical View:

The physical view is quite simple; the view contains little components and follows KISS (keep it simple and stupid). As generic components, the plan is to have a user, computer, router, firewall, internet, API, web browser, and database. As the projects progress, the physical view may change based on what other components that we have failed to add at this time of writing.

The user connects to the computer, which in turn connects to the router. The router goes through the firewall and connects to the internet. After, the flow communicates to the APIs and returns the services. Then to access the site and the database, the flow needs to go through another set of firewalls. Then all this information backtracks and sends the information back up to the internet, which is what is displayed on the computer for the user to see.

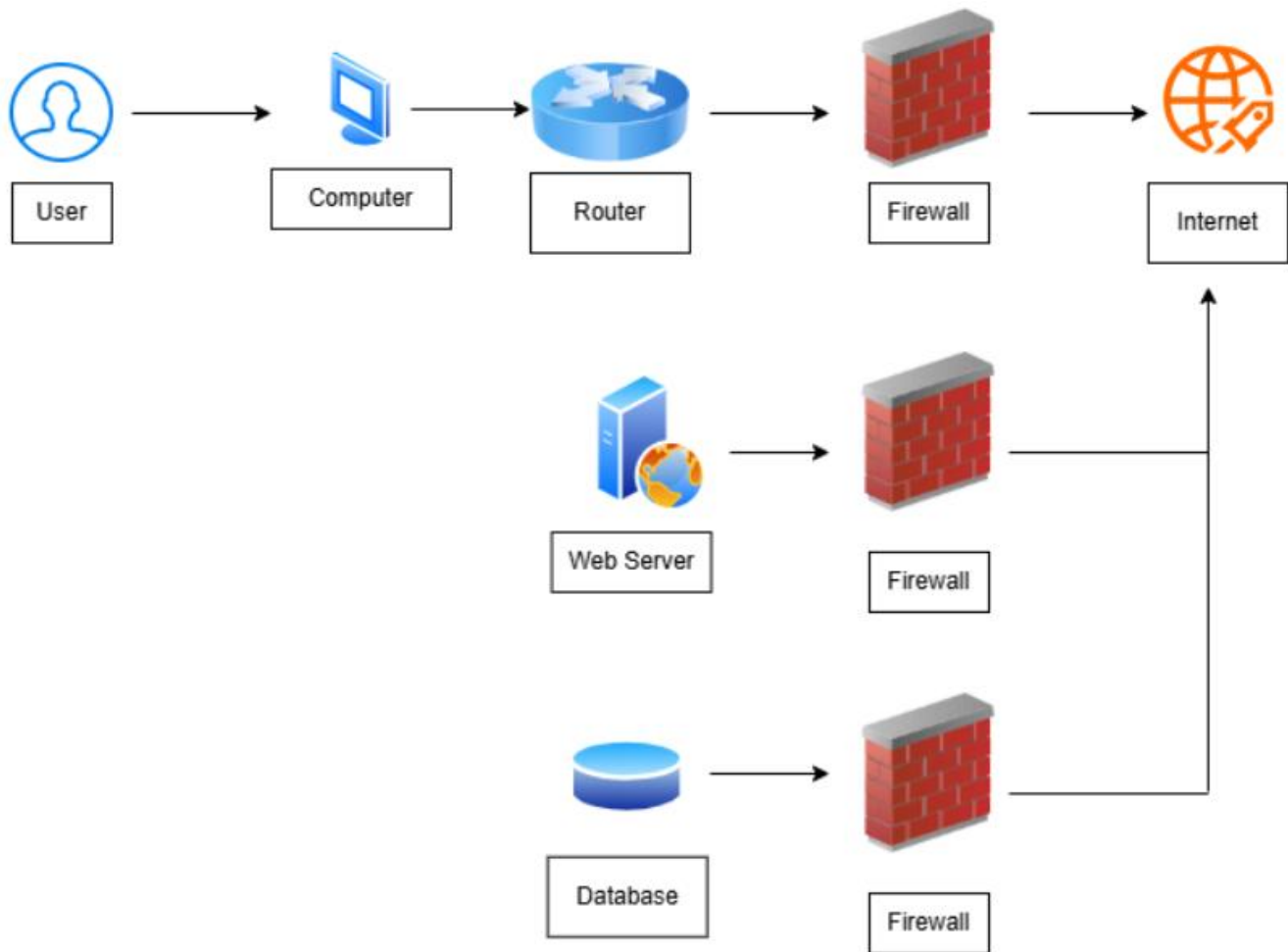


Fig. 1.3. Physical View

N-Tier Architecture Style:

The architecture style Team 7 decided to fit the project the best would be N-Tier architecture. The main reason behind the decision is that the resume web application has three layers: the front-end, back-end, and database (four if including the communication layer from the logical view). The front-end layer should contain all the items that the user can interact with and see with their own eyes. This includes the login page, homepage/dashboard, resume page, favorites tab, and templates tab. The back-end layer contains everything that controls the behind the scenes for the front-end. This includes login confirmation, actual resume generation, downloading resumes, profile management, and so on. The database layer should contain all the information that needs to be recovered for later use: user information, previous resumes, previous resume states, and more.

As seen in the figure below, each component in each layer relates to one another. The front end contains the pages, the back end contains the services, and the databases contain the libraries. For example, if the user were to access the resume templates and select one from the library, it would start



off at the view and edit page. That would then connect to the template service at the back end, then the template library in the database layer. The template library and objects would then be retrieved and sent back to the view and edit page for the user to see.

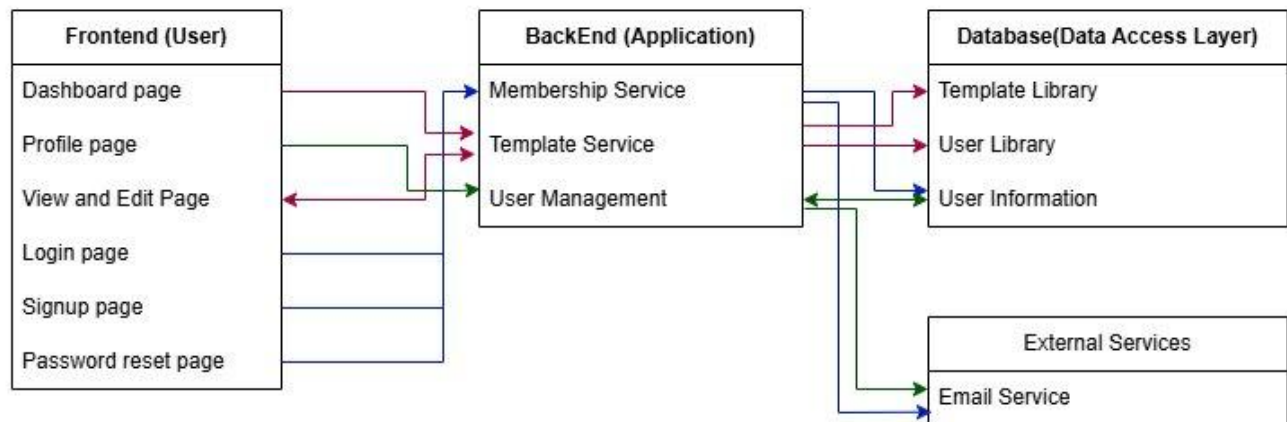


Fig 1.4. N-Tier Architecture Style

UML Diagram:

There are four generic classes that Team 7 has brainstormed. These are the user class, dashboard class, template class, and library class. The user class is a generic class that has anything that corresponds to the user's account/profile information. The dashboard is the main page that contains everything the user sees. There are not a lot of fields or methods within the dashboard due to that reason. The library contains history as well as a template library. It's the main hub where the user retrieves documents/resumes. The templates class is the class where we need to determine what goes into the resume and what is output. Most of the fields are textual fields that rely on user input. As for methods, more can/will be added later as the project progresses.

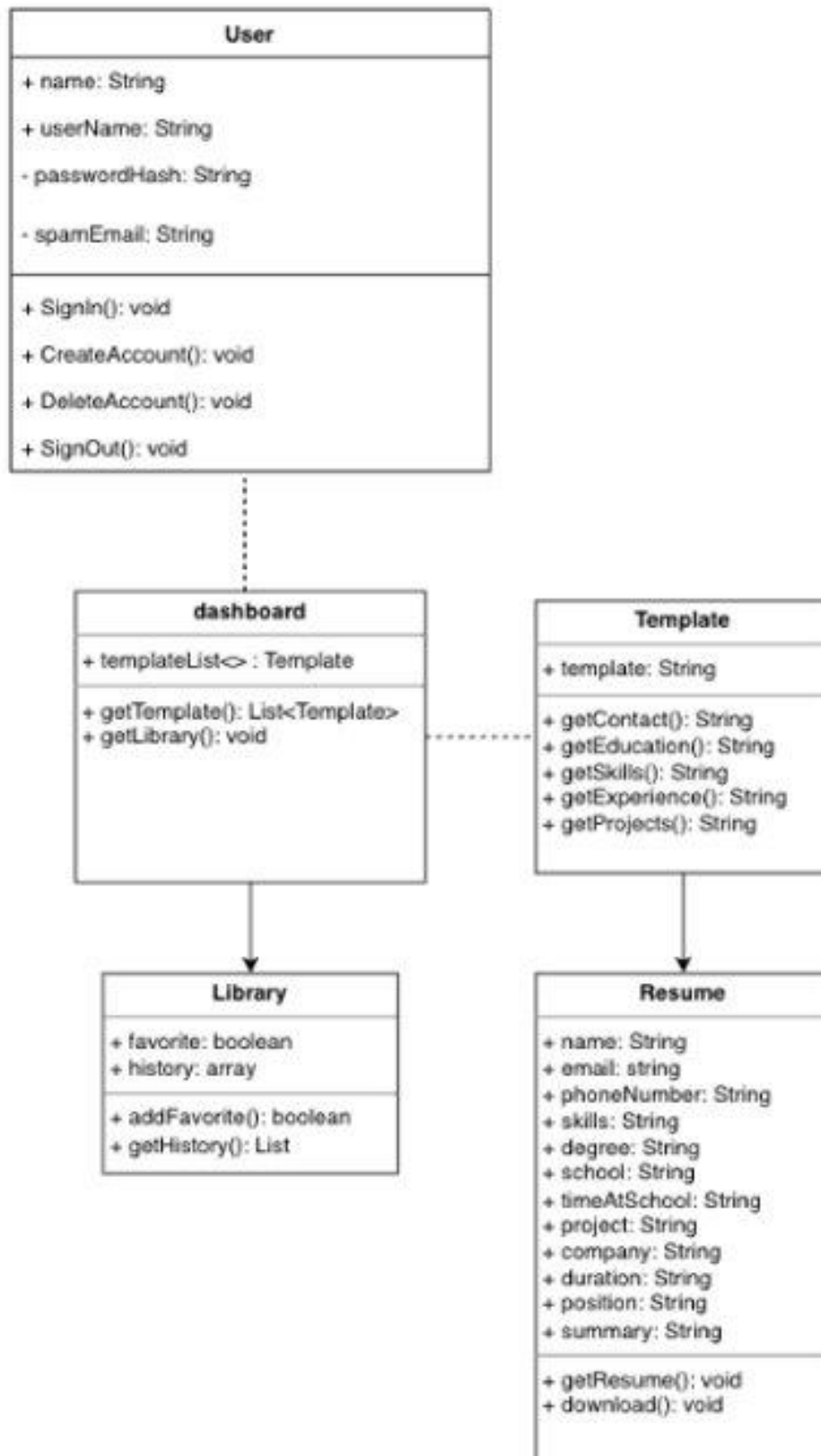


Fig 1.5. UML Diagram



3. SYSTEM MODELING

The following diagrams showcase the system modeling of the project. They will show a select few use cases that will show the diagrams, their expanded use cases, and the sequence diagrams.

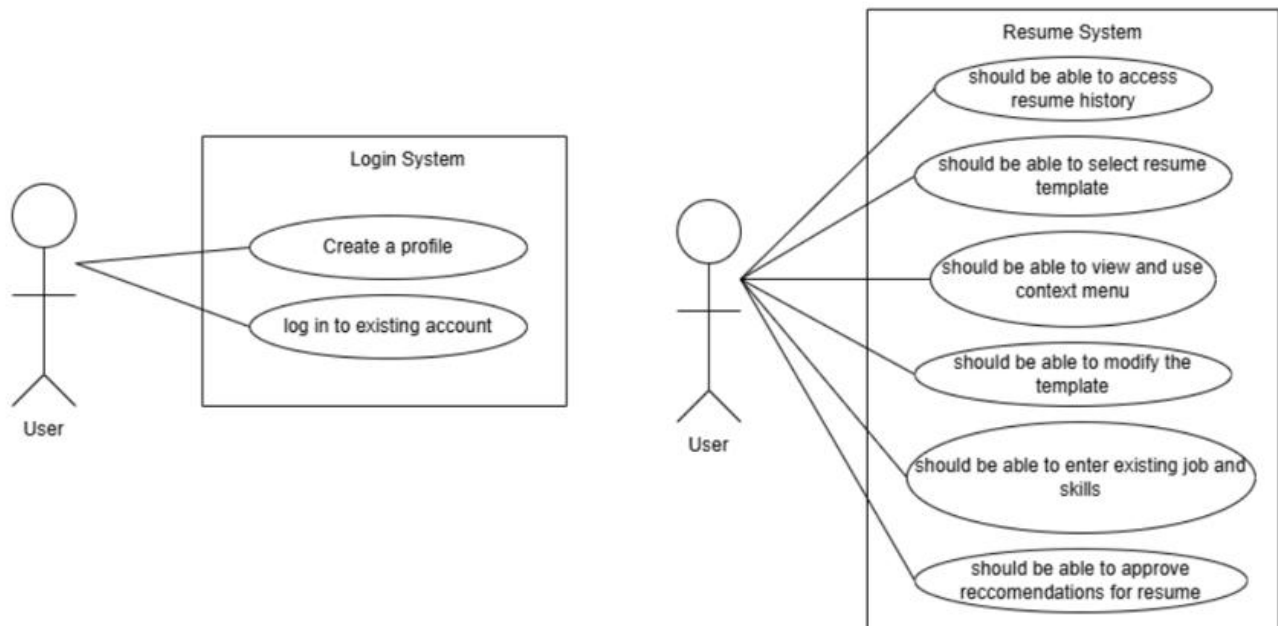


Fig. 2.1. Use Case Diagram

The above diagram shows two systems: the login and resume system. Within the login system, the user should be able to create a profile or log into an existing account. On the other hand, the resume system contains more use cases and complexities. The user should be able to: access resume history, select resume template, view and use context menu (goes more in depth), modify the template (goes more in depth), enter existing job and skills, and approve recommendations to resume. For the context menu and template modification, these are generic groupings that contain more use cases. These can be found in Section 1.

The next few figures will display the use cases, the high-level expansion, expanded use case, and their relevant sequence diagrams.



UC 1: User should be to create a profile

- Pre-condition: They are already on the signup page

Actor: User	System: Login System
	0. system displays signup option
1. TUCBW the user inputs their email and password	2. The system creates the account
5. TUCEW the user sees successful account creation	

- Post-condition: System redirects to the login page

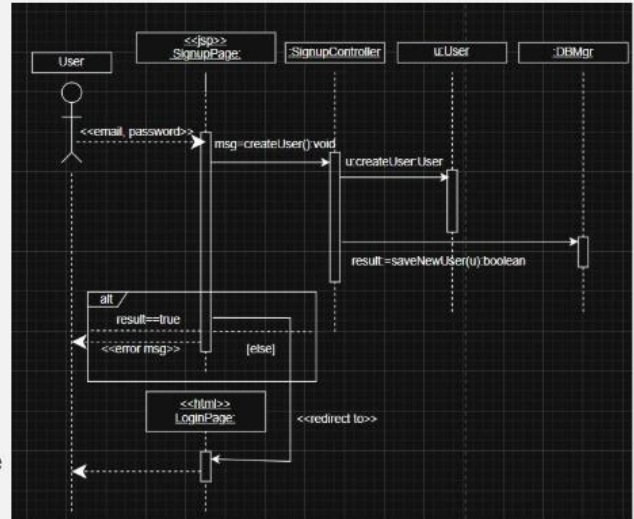


Fig. 2.2. UC 1 Diagrams

UC 2: User should be to login to

- Pre-condition: They should have already signed up and created an account

Actor: User	System: Login System
	0. system displays login option
1. TUCBW the user inputs their email and password	2. The system should authenticate and login the user
5. TUCEW the user sees successful account login	

- Post-condition: System redirects to the dashboard page

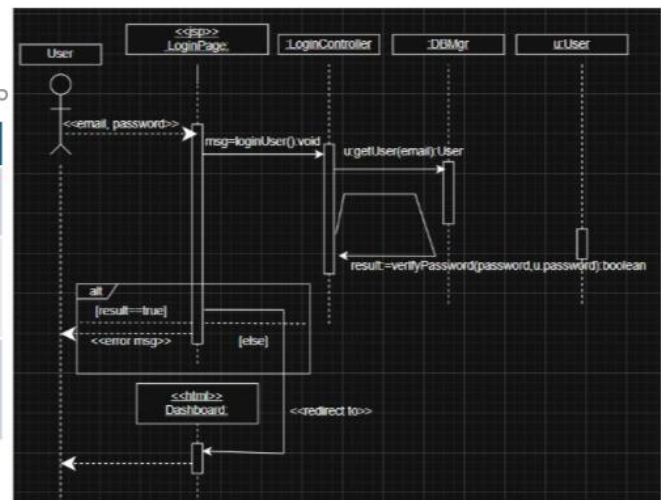
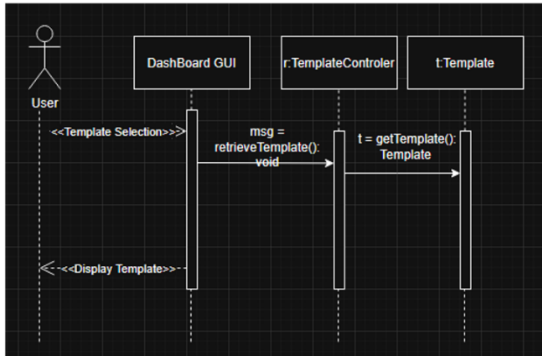


Fig. 2.3 UC 2 Diagrams



UC 4: User should be able to select resume template

Precondition: The user is logged in and plans to create a new resume.



High level use cases:

TUCBW the user scrolling down to see available templets

TUCEW the template appearing on user's screen to be used.

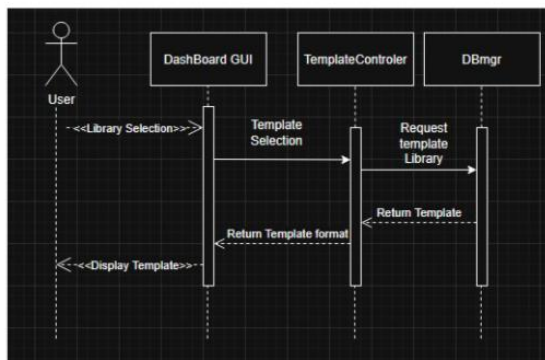
ACTOR: USER	SYSTEM
	0.System displays dashboard on user screen
1. TUCBW User selects a template to use	2. The system creates a copy of template for user to use
3. TUCEW the new template appearing on user's screen to be used.	

Josias

Fig. 2.4. UC 3 Diagrams

UC 4: User should be able to select resume template

Precondition: The user is logged in and plans to create a new resume.



High level cases:

TUCBW the user scrolling down to see available templets

TUCEW the template appearing on user's screen to be used.

ACTOR: USER	SYSTEM
	0.System displays dashboard on user screen
1. TUCBW the user scrolling down to see available templets	2. The system displays all available templates on screen
3. User selects a template to use	4. The system retrieves the template from template library.
5. TUCEW the template appearing on user's screen to be used.	

Josias

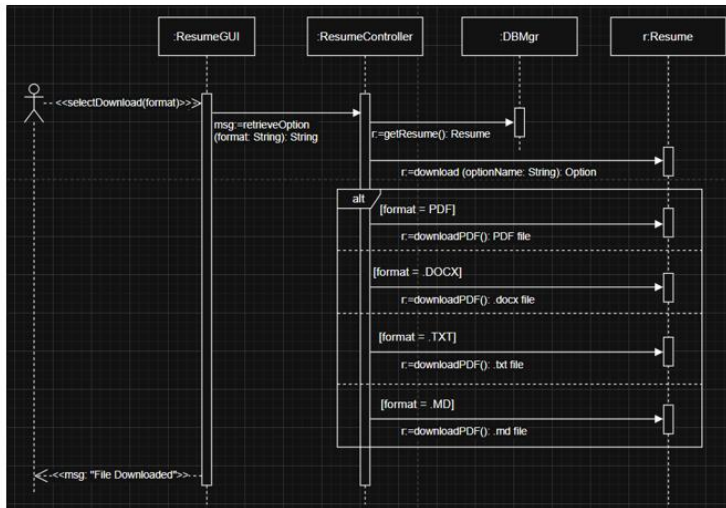
Fig. 2.5. UC 4 Diagrams



UC 6: User should be able to download resume

Pre-condition:
Assumes that the user is on the resume page and has navigated to the context menu

Post-condition:
Resume is downloaded to the user's device



High-Level Use Case:

TUCBW: The user selects the "Download" button

TUCEW: The user sees a message regarding download status

Actor: User	System: Resume System
	0. Resume + context menu are displayed
1. TUCBW the user selects the "Download" button	2. The system displays a list of download options
3. The user selects one of the options	*4. The system runs the selected option
5. TUCEW the user sees message displaying successful file download	

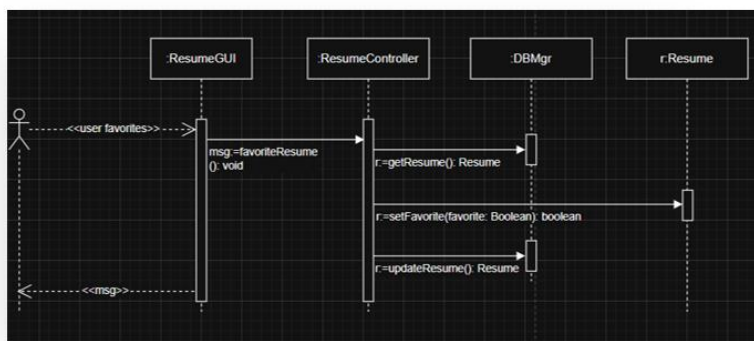
Ivan

Fig. 2.6. UC 6 Diagrams

UC 9: User should be able to favorite resume

Pre-condition:
Assumes the user is already on the resume page and viewing the context menu

Post-condition:
Favorites should include resume "favorited" by the user



High-Level Use Case:

TUCBW: The user selects the "Favorite" button

TUCEW: The user sees the resume being added to "Favorites":

Actor: User	System: Resume System
	0. Resume is displayed
1. TUCBW the user selects "Favorite" resume option	*2. The system adds the file to Favorites
3. TUCEW the user sees the resume being added to "Favorites"	

Ivan

Fig. 2.7. UC 9 Diagrams



UC 7: Fill out resume

Precondition: The user selects desired template

Actor: User	System
	0. System displays resume
1. TUCBW the user putting data into the text boxes	2. The system displays resume with inputted data
3. The user clicks save	3. The system updates the database with new resume and displays final resume to user
4. TUCEW the user views the resume	

High-Level use Case:

TUCBW: the user inputs data

TUCEW: the user views finalized resume

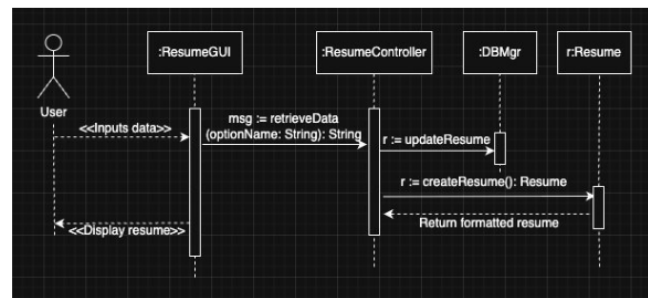


Fig. 2.8. UC 7 Diagrams

UC 8: Approve resume recommendations

Precondition: The user has opened the resume they want to edit/create

Actor: User	System
	0. The system displays to the user the edit screen
1. TUCBW the user inputting data	2. The system saves data and displays to user buzzword recommendations
3. The user clicks on buzzwords	4. The system adds the buzzword to the resume, and displays the changes to user
5. The user saves the resume	6. The system saves resume to database
7. TUCEW the user viewing finalized resume	

High-Level use Case:

TUCBW: the user inputs data

TUCEW: the user views finalized resume

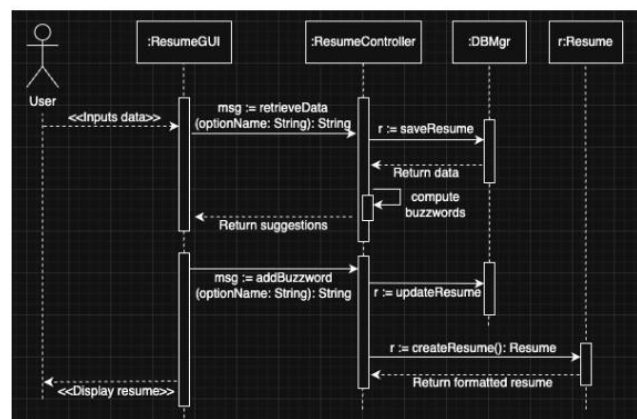


Fig. 2.9. UC 8 Diagrams



4. WIRE FRAMES

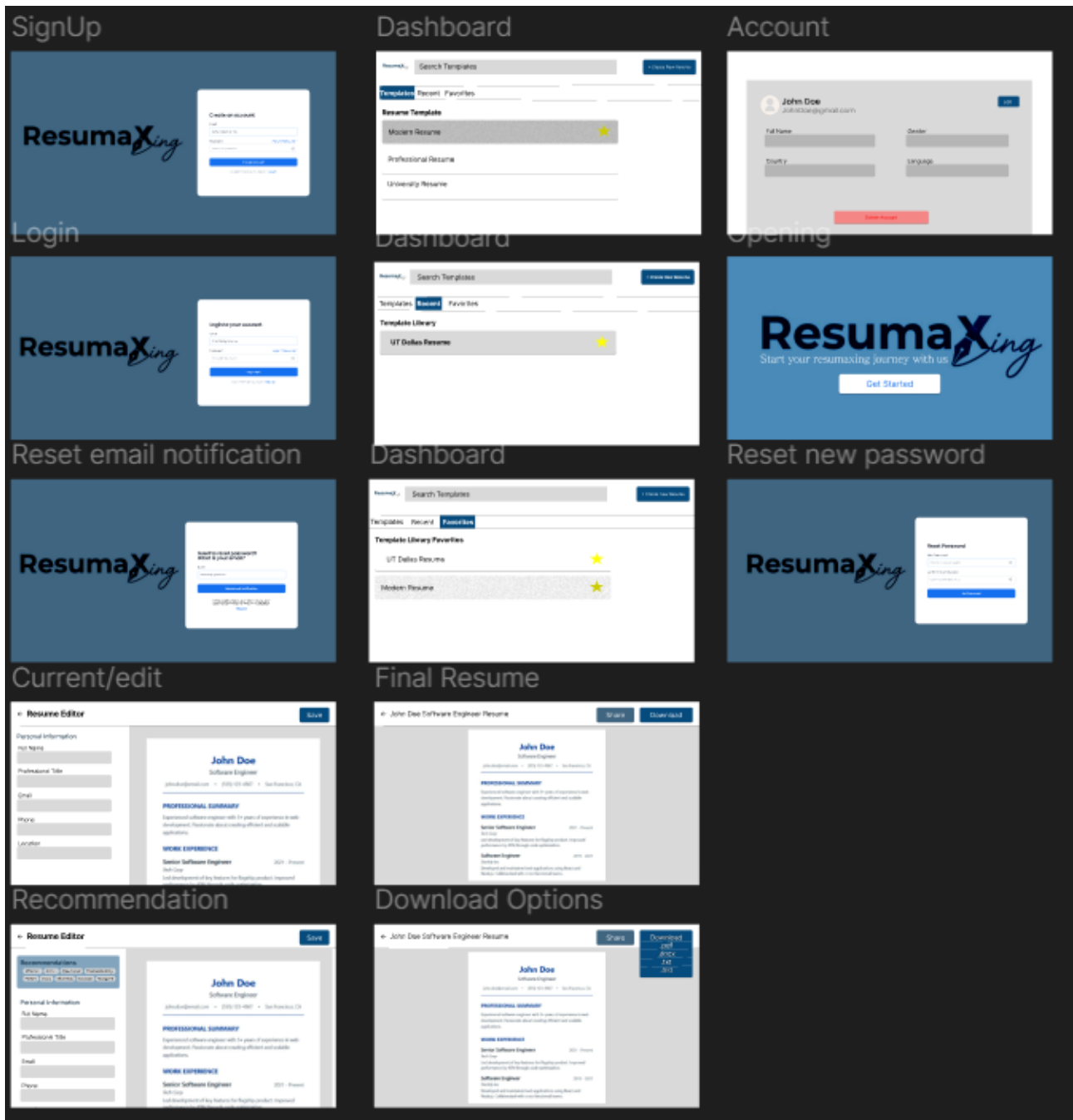


Fig. 4.1. Figma Wireframe ([link to Figma](#))



5. DEVELOPMENT RESEARCH & PLAN

Developmental Methodology:

- Compared two potential methodologies: Agile and Waterfall.
 - o Settled on Agile for its iterative property, this approach allows the team to build/code and then test to ensure that the code works as intended. This continuous workflow will create quality code that ensures little to no errors. The flexibility provided has less constraints in the workflow and allows for a loop like structure that gives the team the ability to go back and plan again.
 - o Waterfall was not selected as it was too constrained by its structure. The sequential flow that requires testing at each level did not resonate with the proposed project idea. The fact of having to check at each level to get everything perfectly done and not coming back up to it was not sufficient and ideal for the project workflow.
- In addition to the Agile methodology, we included the MVC Framework for the designing of the design class diagram. The MVC Framework shows the connections between the model (M), view (V), the controller (C), the database, and the end user.
 - o The reason for using this framework goes as follows:
 - From the expanded use case diagrams, it was decided that controllers will act as a bridge to communicate between the GUIs and database and classes.
 - The classes used and listed out in the expanded use case diagrams will also communicate with other classes within the system.
 - o From the explained reasons above, the MVC Framework was used as it showed a consistent yet organized flow that represented the idea of the project well enough. It perfectly represented the CRUD (create, read, update, delete) operations as well provided flexible connections between each module.

Technology Stack:

- Front-End: HTML, CSS, JavaScript
- Back-End: Python w/ Flask
- Database: Supabase

Collaboration & Version Control:

- Git & GitHub for version control and repository management
- Peer reviews are done to maintain quality assurance and knowledge sharing

Testing Strategies:

- Functional Testing: components and integration
- Non-Functional Testing: performance and usability

Testing Types:



- Manual Testing: we planned to test our use cases by hand allowing us to pinpoint what cases needed adjustment. Using this method, we were able to test the edge cases allowing for complete coverage of the app usability
 - Due to the nature of the application/project, most of the test cases are based on the user's mouse click input (which relies on event detectors). In return, the test cases do not rely on third-party software to write and test unique test cases.
 - However, test cases that may use third-party software are those involved with the login/sign-up for the users. Although these test cases can be generated through outside means, the team will still manually create their own test cases to test on their own machines.
- Unit Testing: the team will test functionality of the methods, classes, and functions in isolation to ensure that they work on their own
 - Through the usage of Git and GitHub, each member of the team will be able to edit the code on their own. In addition, they will be able to add their own functionalities all the while having the ability to test them.
 - By effectively working in isolation, each member can test their own units of code without having to inadvertently damage another member's workspace or causing confusing changes.
 - By ensuring that each unit works by itself, the team can confidently push their code changes to the main branch since the unit is able to work on its own.
- Integration Testing: the team will put all the units together and test to ensure that the code functions as intended
 - Once again, through the nature of Git and GitHub, the members of the team can merge their respective isolated, unit-tested, functionalities together to assemble the main project.
 - The team will then be able to test the entire project and ensure that all models that were pushed have no conflicts and can easily interact with each other. Furthermore, if another member were to add another module of their choosing (one that is properly unit-tested), other members of the team will have the ability to test the entire product to check for any conflicts or errors that may break the system.

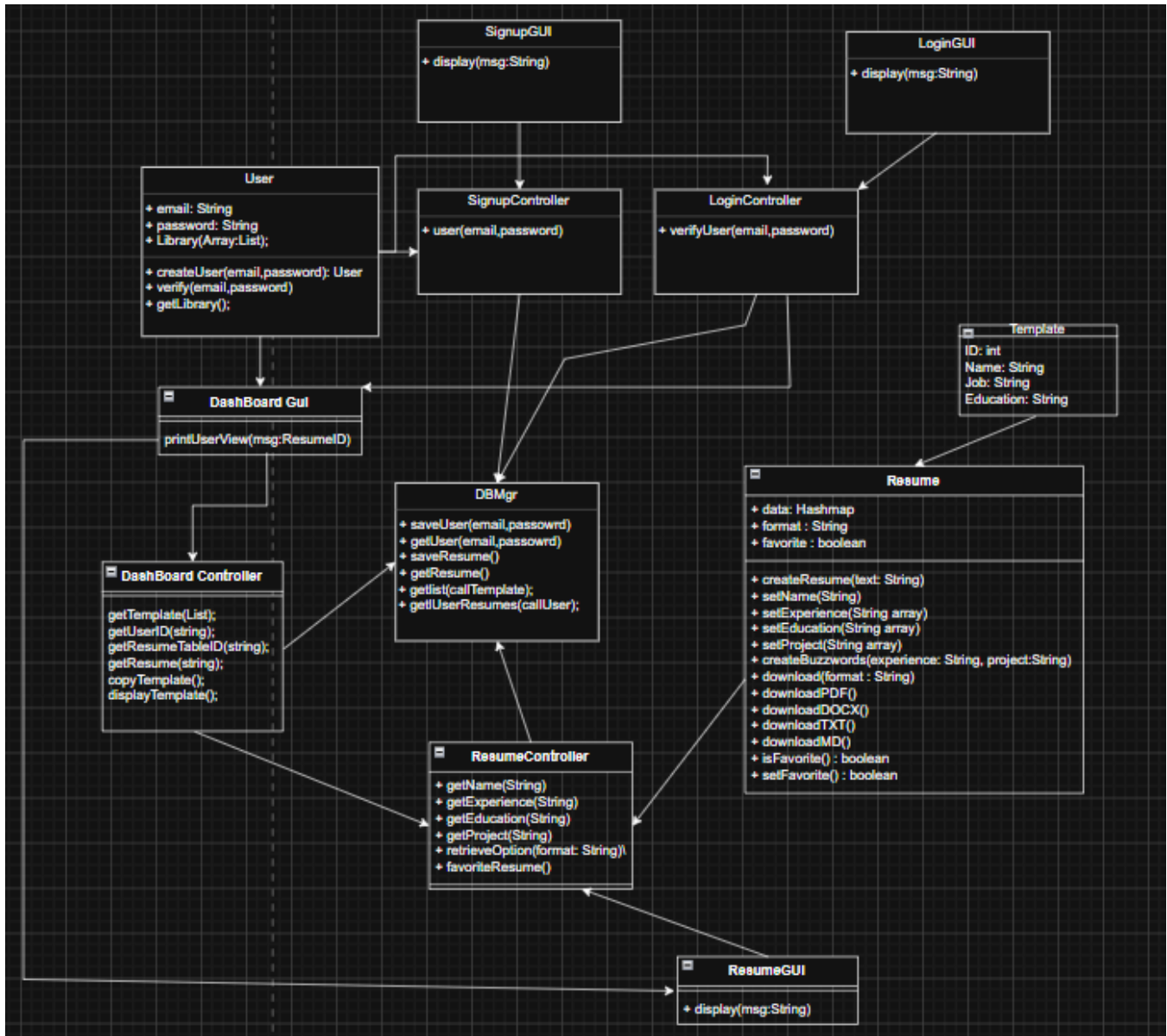


Fig 5.1. Design Class Diagram

Figure 5.1. showcases the design class diagram. It essentially compiles everything that was created in the sequence diagrams from earlier and throws them into a “UML-esque” format. These classes are split up into three sections: Front-End, Back-End, and Database. This will help with the building process of the project by simply stating which class belongs in which section. It also helps with understanding which class they have, what methods they use, and how they connect with each other. The individual classes will be tested through unit testing while the project will be tested with integration testing.



6. IMPLEMENTATION

Link to the GitHub: [Hanna-Kellenberger/git-assignment](https://github.com/Hanna-Kellenberger/git-assignment)

Deployment Strategy: Using local host (located on port 3000) as the host server, the app is built using the following:

- Front-End: HTML, CSS, JavaScript
- Back-End: Python, Flask, Ollama
- Database: Supabase

The design follows the N-Tier Architecture Style by utilizing the front-end, back-end, and database aspects of the application.



7. SOFTWARE TESTING

The following images will showcase the functional test cases:

UC1: TC ID	Scenario	Email	Password	Expected Result	Actual Result
1	successful account creation	Vaild Email ID: Test@gmail.com	8-18 charecters, 1 capital, 1 special charecter - Testpassword@	Message "Account Successfully Created"	Just directly redirts to login page
2	No email input	empty	N/A	Message "Please Input an Email"	Error message will show "Please fill out this field"
3	No password input	Test@gmail.com	empty	Message "Please Input a Valid Password - it should be 8-18 charecteristics, 1 capital letter, 1 special charecter "	Error message will show "Please fill out this field"
4	Wrong password input - less than 8 charecters	Test@gmail.com	Test@	Message "Please Input a Valid Password - it should be 8-18 charecteristics, 1 capital letter, 1 special charecter "	Error message will show "Signup failed"
5	Wrong password input - more than 18 charecters	Test@gmail.com	Testpasswordclassss@	Message "Please Input a Valid Password - it should be 8-18 charecteristics, 1 capital letter, 1 special charecter "	Error message will show "Signup failed"
6	Wrong password input - no capital letter	Test@gmail.com	testpassword@	Message "Please Input a Valid Password - it should be 8-18 charecteristics, 1 capital letter, 1 special charecter "	Error message will show "Signup failed"
7	Wrong password input - no special charecter	Test@gmail.com	Testpassword	Message "Please Input a Valid Password - it should be 8-18 charecteristics, 1 capital letter, 1 special charecter "	Error message will show "Signup failed"

UC2: TC ID	Scenario	Email	Password	Expected Result	Actual Result
1	successful login	Vaild Email ID same as account creation: Test@gmail.com	Valid Password same as account creation - Testpassword@	Redirect to the Dashboard Page	Redirect to the Dashboard Page
2	Wrong email input	Invalid Email ID not same as account creation: Invalid@gmail.com	N/A	Message "Invalid Email or Password Please Try Again"	Message "Login failed."
3	No email input	empty	N/A	Message "Invalid Email or Password Please Try Again"	Error message will show "Please fill out this field"
4	No password input	Test@gmail.com	empty	Message "Invalid Email or Password Please Try Again"	Error message will show "Please fill out this field"
5	Wrong password input - less than 8 charecters	Test@gmail.com	Test@	Message "Invalid Email or Password Please Try Again"	Message "Login failed."
6	Wrong password input - more than 18 charecters	Test@gmail.com	Testpasswordclassss@	Message "Invalid Email or Password Please Try Again"	Message "Login failed."
7	Wrong password input - no capital letter	Test@gmail.com	testpassword@	Message "Invalid Email or Password Please Try Again"	Message "Login failed."
8	Wrong password input - no special charecter	Test@gmail.com	Testpassword	Message "Invalid Email or Password Please Try Again"	Message "Login failed."



Account Testcases	Scenario	Account icon	logout button	Expected Result	Actual Result
1	See account details	Pressed	N/A	Dropdown menu with email and logout button is visible	Dropdown menu with email and logout button is visible
2	Account not pressed	Not pressed	N/A	Remain on dashboard page	Remain on dashboard page
3	Successfully logged out	Not pressed	Pressed	Page redirects to login page	Page redirects to login page
4	No logout	Not pressed	Not pressed	Just remain on dashboard page with dropdown menu visible	Just remain on dashboard page with dropdown menu visible

Forget Password Testcases	Scenario	Forget Password? Button	Email	Password	Reset Link	Request new link	Expected Result	Actual Result
1	Forget password button pressed	pressed	N/A	N/A	N/A	N/A	Page redirects to send reset link page	Page redirects to send reset link page
2	Forget password button not pressed	not pressed	N/A	N/A	N/A	N/A	Page remains on login page	Page remains on login page
3	Reset password link is sent	N/A	The user's email they used to create account Test@gmail.com	N/A	N/A	N/A	Link will be sent to inbox	Link will be sent to inbox
4	Reset password link page email is not input	N/A	No input	N/A	N/A	N/A	Error message will show "Please fill out this field"	Error message will show "Please fill out this field"
5	Reset link is clicked	N/A	N/A	N/A	pressed	N/A	If link is not expired then link will redirect to the reset password page	If link is not expired then link will redirect to the reset password page
6	Reset link is clicked	N/A	N/A	N/A	pressed	N/A	If link is expired then an error message pops up that says "link has expired resend link"	If link is expired then the error message pops up that says "Email link is invalid or has expired Request a new link"
7	Request new link is clicked	N/A	N/A	N/A	N/A	pressed	Page redirects to send reset link page	Page redirects to send reset link page
8	Request new link is not clicked	N/A	N/A	N/A	N/A	not pressed	Remain on the get started page with the error message	Remain on the get started page with the error message
9	new password input	N/A	N/A	Testpasswordclasss@	N/A	N/A	Resets password and redirects to login page	Resets password and redirects to login page
10	Wrong password input - no special character	N/A	N/A	Testpassword	N/A	N/A	Message "Please Input a Valid Password - It should be 8-16 characteristics, 1 capital letter, 1 special character "	Message "Please Input a Valid Password - It should be 8-16 characteristics, 1 capital letter, 1 special character "

Fig 7.1. Functional Test Cases (UC1, UC2, Account, Forget Password)

UC#3				
TC ID				
UC#3	Scenario	Input	Expected result	Actual result
TC 1	Successfully opening of Past Resume	User Resume ID	Opening Resume with past information	Opening Resume with past information
TC 2	Nothing occurs	None	Nothing Changes	nothing changes



TC ID UC #4	Scenario	Input	Expected Result	Actual Result	
TC 1	Successfully opening of New Template	Template ID	Opening New Template	Opening New Template	
TC 2	Nothing occurs	NONE	nothing changes	nothing changes	

Fig 7.2. Functional Test Cases & Traceability Matrix (UC3, UC4)

UC 5:								
TC ID	Scenario	Context Menu Dropdown	Download	Share	Rename	Favorite	Expected Result	Actual Result
5.1	Context Menu Selected	Context Menu is clicked on	None	None	None	None	Context Menu dropdown is shown	Context Menu Dropdown is shown
5.2	Context Menu not Selected	Context Menu is not clicked on	None	None	None	None	Context Menu dropdown is not shown	Context Menu Dropdown is not shown
5.3	Download is Selected	Context Menu is clicked on	Download is clicked on	None	None	None	Download dropdown is shown	Download dropdown is shown
5.4	Download is not Selected	Context Menu is clicked on	Download is not clicked on	None	None	None	Download dropdown is not shown	Download dropdown is not shown
5.5	Share is Selected	Context Menu is clicked on	None	Share is clicked on	None	None	Share dropdown is shown	Share dropdown is shown
5.6	Share is not Selected	Context Menu is clicked on	None	Share is not clicked on	None	None	Share dropdown is not shown	Share dropdown is not shown
5.7	Rename is Selected	Context Menu is clicked on	None	None	Rename is clicked on	None	User can change resume name	User can change resume name
5.8	Rename is not Selected	Context Menu is clicked on	None	None	Rename is not clicked on	None	User cannot change resume name	User cannot change resume name
5.9	Favorite is Selected	Context Menu is clicked on	None	None	None	Favorite is clicked on	Resume is added to favorites	Resume is added to favorites
5.10	Favorite is not Selected	Context Menu is clicked on	None	None	None	Favorite is not clicked on	Resume is not added to favorites	Resume is not added to favorites
UC 6:								
TC ID	Scenario	Download Dropdown	PDF	DOCX	TXT	MD	Expected Result	Actual Result
6.1	User quits mid selection	User quits	None	None	None	None	Resume is not downloaded	Resume is not downloaded
6.2	Successful Download	Download Dropdown is selected	Download as a PDF is clicked on	None	None	None	Resume is downloaded as a PDF	Resume is downloaded as a PDF
6.3	Successful Download	Download Dropdown is selected	None	Download as a DOCX is clicked on	None	None	Resume is downloaded as DOCX	Resume is downloaded as DOCX
6.4	Successful Download	Download Dropdown is selected	None	None	Download as a TXT is clicked on	None	Resume is downloaded as a TXT	Resume is downloaded as a TXT
6.5	Successful Download	Download Dropdown is selected	None	None	None	Download as an MD is clicked on	Resume is downloaded as a MD	Resume is downloaded as a MD

Fig 7.3. Functional Test Cases (UC5, UC6)



Use Case 7								
TC ID	Scenario	Contact Information	Summary	Education	Job Experience	Skills	Expected Result	Actual Result
TC1	Successfully create resume	1112223333	This is a brief summary used for testing.	University of Texas at Dallas Bachelors of Science	McDonalds	html, css, javascript	Create resume	Resume is saved to recent tab
TC2	User leaves a text box blank	(empty)	This is a brief summary used for testing.	University of Texas at Dallas Bachelors of Science	McDonalds	html, css, javascript	Error: Contact information is required	Email is empty Phone is empty
TC3	User leaves a text box blank	1112223333	(empty)	University of Texas at Dallas Bachelors of Science	McDonalds	html, css, javascript	Error: Summary is required	Summary is checked but empty
TC4	User leaves a text box blank	1112223333	This is a brief summary used for testing.	(empty)	McDonalds	html, css, javascript	Error: Education is required	Education: Degree is empty Education: School is empty Education: Dates is empty
TC5	User leaves a text box blank	1112223333	This is a brief summary used for testing.	University of Texas at Dallas Bachelors of Science	(empty)	html, css, javascript	Error: Job experience is required	Experience: Job Title is empty Experience: Company is empty Experience: Dates is empty
TC6	User leaves a text box blank	1112223333	This is a brief summary used for testing.	University of Texas at Dallas Bachelors of Science	McDonalds	(empty)	Error: Skills is required	Skills is checked but no skills added
TC7	User does not fill out resume	(empty)	(empty)	(empty)	(empty)	(empty)	Error: Please input values	Shows an error for each empty text box

User Case 8					
TC ID	Scenario	Skill	Job Experience	Expected Result	Actual Result
TC1	User inputs Skills and Job Experience	html, css, java	McDonalds Worked as a cashier	Generate suggestions	Generates skills in the skills box. click to add to resume skills section.
TC2	User is missing skills	(empty)	McDonalds Worked as a cashier	Generate suggestions based off of skills	Generates skills in the skills box. click to add to resume skills section.
TC3	User is missing job experience	html, css, java	(empty)	Generate suggestions based off of job experience	Generates skills in the skills box. click to add to resume skills section.
TC4	No skill or job experience	(empty)	(empty)	Does not generate anything	Fill in your job title, experience, or existing skills before requesting suggestions

Fig 7.4. Functional Test Cases (UC7, UC8)



UC 9:							
TC ID	Scenario	Favorite	Expected Result	Actual Result			
9.1	Successful removal from Favorites	Favorite is clicked on	Resume is removed from Favorites	Resume is removed from Favorites			
9.2	Successful addition to Favorites	Favorite is clicked on	Resume is added to Favorites	Resume is added to Favorites			
9.3	User quits mid selection	User quits	Nothing changes	Nothing changes			
UC 10:							
TC ID	Scenario	Share Dropdown	Link	Embed	Email	Expected Result	Actual Result
10.1	User quits mid selection	Shared Dropdown is not selected	None	None	None	Resume is not shared	Resume is not shared
10.2	Shared Successfully	Shared Dropdown is selected	Share as a link is clicked on	None	None	Link to resume is available	Link to resume is available
10.3	Shared Successfully	Shared Dropdown is selected	None	Share as an embed is clicked on	None	Resume embed is available	Resume embed is available
10.4	Shared Successfully	Shared Dropdown is selected	None	None	Share through email is clicked on	Sharing via email is available	Sharing via email is available

Fig 7.5. Functional Test Cases (UC9, UC10)

UC 11:							
TC ID	Scenario	Font Size	Font	Font Color	Font Format	Expected Result	Actual Result
11.1	Increase Font Size via click	"+" is clicked	N/A	N/A	N/A	Font size increments by one	Not Implemented
11.2	Decrease Font Size via click	"-" is clicked	N/A	N/A	N/A	Font size decrements by one	Not Implemented
11.3	Increase Font via input	12 → 24	N/A	N/A	N/A	Font size increases to desired value	Not Implemented
11.4	Decrease Font via Input	24 → 12	N/A	N/A	N/A	Font size decreases to desired value	Not Implemented
11.5	User quits when selecting font size	User quits	N/A	N/A	N/A	Font size resorts to current font	Not Implemented
11.6	Font Size Max Reached (click)	"+" is clicked	N/A	N/A	N/A	Font size stays at max value	Not Implemented
11.7	Font Size Min Reached (click)	"-" is clicked	N/A	N/A	N/A	Font size stays at min value	Not Implemented
11.8	Font Size Out of Range (via input)	input < 6 or input > 36	N/A	N/A	N/A	Error Message: Font Size Out of Bounds	Not Implemented
11.9	Font is Chosen	N/A	Times New Roman	N/A	N/A	Desired font is picked	Not Implemented
11.1	User quits when selecting font	User quits	Nothing	N/A	N/A	Font resorts to current font	Not Implemented
11.11	Font Color is Chosen	N/A	N/A	Blue	N/A	Desired font color is picked	Not Implemented
11.12	User quits when selecting font color	User quits	N/A	Nothing	N/A	Font color resorts to current color	Not Implemented
11.13	Font Format is Selected	N/A	N/A	N/A	Bold	Reflected changes according to format attribution	Not Implemented
11.14	Font format is not selected	N/A	N/A	N/A	Nothing	Format changes according to format attribution	Not Implemented
11.15	Font is not changed	Nothing	Nothing	Nothing	Nothing	Nothing changes	Not Implemented

Fig 7.6. Functional Test Cases (UC11)



UC 12:							
TC ID	Scenario	Rename	Expected Result	Actual Result			
12.1	Successful rename	Rename is selected	Name is changed to input	Name is changed to input			
12.2	User quits when renaming	Rename is not selected	Name stays the same; resorts to current name	Name is set to whatever the user last typed in			
12.3	New name is empty		Name is set to default value	Name stays empty			
12.4	New name is too long	input > 100 characters	Error Message: Name too long, must be less than 100 characters	No length bound to name			
UC 13:							
TC ID	Scenario	Alignment Dropdown	Left	Center	Right	Expected Result	Actual Result
13.1	Successful Alignment	Alignment is clicked on	Left is clicked on	N/A	N/A	Left alignment of selected section	Not Implemented
13.2	Successful Alignment	Alignment is clicked on	N/A	Center is clicked on	N/A	Center alignment of selected section	Not Implemented
13.3	Successful Alignment	Alignment is clicked on	N/A	N/A	Right is clicked on	Right alignment of selected section	Not Implemented
13.4	User quits when selecting alignment	User quits	N/A	N/A	N/A	Nothing changes	Not Implemented

Fig 7.7. Functional Test Cases (UC12, UC13)

UC 14:					
TC ID	Scenario	Reorder via input	Reorder via click and drag	Expected Result	Actual Result
14.1	Successful Order Swap	4 → 1	N/A	Order is swapped to numbered location	Not Implemented
14.2	Successful Order Swap	N/A	Click and Drag	Order is swapped to dragged location	Order is swapped to dragged location
14.3	Order Number Out of Range	input < 0 or input > number of sections	N/A	Error Message: Section Number Out of Bounds	Not Implemented
14.4	User quits mid input	User quits	N/A	Section is placed in original/current position	Not Implemented
14.5	User quits mid click and drag	N/A	User quits	Section is placed in original/current position	Section is placed in original/current position
UC 15:					
TC ID	Scenario	Section	Expected Result		Actual Result
15.1	Successful Section Addition	Add Section is clicked on	A new section is added		Not Implemented
15.2	Successful Section Removal	Remove Section is clicked on	A section is removed		A section is removed
15.3	Unsuccessful Section Removal	num sections = 1	Error Message: Cannot have less than 1 section		All sections can be removed, except for personal information
15.4	Unsuccessful Section Addition	num sections = 5	Error Message: Cannot have more than 5 sections		Sections cannot be added but reinstated
15.5	User quits mid selection	User quits	Nothing Changes		Nothing changes

Fig 7.8. Functional Test Cases (UC14, UC15)



The following images will showcase the functional and non-functional test case traceability matrices:

UC1: TC ID	Scenario	Functional Requirements	Non-Functional Requirements
1	successful account creation	FR1.1, FR1.2, FR1.3	NFR1.5, NFR3.5, NFR7.5
2	No email input	FR1.2	NFR3.5, NFR7.5
3	No password input	FR1.3	NFR3.5, NFR7.5
4	Wrong password input - less than 8 charecters	FR1.3	NFR3.5, NFR7.5
5	Wrong password input - more than 18 charecters	FR1.3	NFR3.5, NFR7.5
6	Wrong password input - no capital letter	FR1.3	NFR3.5, NFR7.5
7	Wrong password input - no special charecter	FR1.3	NFR3.5, NFR7.5
UC2: TC ID	Scenario	Functional Requirements	Non-Functional Requirements
1	successful login	FR2.1, FR2.2, FR2.3	NFR2.5, NFR2.6, NFR3.5, NFR7.5
2	Wrong email input	FR2.2	NFR3.5, NFR7.5
3	No email input	FR2.2	NFR3.5, NFR7.5
4	No password input	FR2.3	NFR3.5, NFR7.5
5	Wrong password input - less than 8 charecters	FR2.3	NFR3.5, NFR7.5
6	Wrong password input - more than 18 charecters	FR2.3	NFR3.5, NFR7.5
7	Wrong password input - no capital letter	FR2.3	NFR3.5, NFR7.5
8	Wrong password input - no special charecter	FR2.3	NFR3.5, NFR7.5

Fig 7.9. Functional and Non-Functional Traceability Matrices (UC1, UC2)



UC 5 TC ID	Scenario	Functional Req.	Non-Functional Req.
5.1	Context Menu Selected	FR-5.1, FR-5.2, FR-5.3, FR-5.4, FR-5.5	NFR-3.5, NFR-7.5
5.2	Context Menu not Selected	FR-5.1, FR-5.2, FR-5.3, FR-5.4, FR-5.5	NFR-3.5, NFR-7.5
5.3	Download is Selected	FR-5.1, FR-5.2, FR-5.3, FR-5.4, FR-5.5	NFR-3.5, NFR-7.5
5.4	Download is not Selected	FR-5.1, FR-5.2, FR-5.3, FR-5.4, FR-5.5	NFR-3.5, NFR-7.5
5.5	Share is Selected	FR-5.1, FR-5.2, FR-5.3, FR-5.4, FR-5.5	NFR-3.5, NFR-7.5
5.6	Share is not Selected	FR-5.1, FR-5.2, FR-5.3, FR-5.4, FR-5.5	NFR-3.5, NFR-7.5
5.7	Rename is Selected	FR-5.1, FR-5.2, FR-5.3, FR-5.4, FR-5.5	NFR-3.5, NFR-7.5
5.8	Rename is not Selected	FR-5.1, FR-5.2, FR-5.3, FR-5.4, FR-5.5	NFR-3.5, NFR-7.5
5.9	Favorite is Selected	FR-5.1, FR-5.2, FR-5.3, FR-5.4, FR-5.5	NFR-3.5, NFR-7.5
5.10	Favorite is not Selected	FR-5.1, FR-5.2, FR-5.3, FR-5.4, FR-5.5	NFR-3.5, NFR-7.5
UC 6 TC ID	Scenario	Functional Req.	Non-Functional Req.
6.1	User quits mid selection	FR-6.1, FR-6.2, FR-6.3, FR-6.4	NFR-2.8, NFR-3.5, NFR-5.5, NFR-7.5
6.2	Successful Download	FR-6.1, FR-6.2, FR-6.3, FR-6.4	NFR-2.8, NFR-3.5, NFR-5.5, NFR-7.5
6.3	Successful Download	FR-6.1, FR-6.2, FR-6.3, FR-6.4	NFR-2.8, NFR-3.5, NFR-5.5, NFR-7.5
6.4	Successful Download	FR-6.1, FR-6.2, FR-6.3, FR-6.4	NFR-2.8, NFR-3.5, NFR-5.5, NFR-7.5
6.5	Successful Download	FR-6.1, FR-6.2, FR-6.3, FR-6.4	NFR-2.8, NFR-3.5, NFR-5.5, NFR-7.5

Fig 7.10. Functional and Non-Functional Traceability Matrices (UC5, UC6)

UC7 TC ID	Scenario	Functional Req.	Nonfunctional Req.	UC8 TC ID	Scenario	Functional Req.	Nonfunctional Req.
TC1	Successfully create resume	FR-7.1, FR-7.2, FR-7.3, FR-7.4, FR-8.1, FR-8.2, FR-8.3, FR-8.4	NFR-2.7, NFR-3.5, NFR-7.5	TC1	User inputs Skills and Job Experience	FR-7.3, FR-8.1,	NFR-2.7, NFR-3.5
TC2	User leaves a text box blank	FR-7.1, FR-7.2, FR-7.3, FR-7.4, FR-8.1	NFR-2.7, NFR-3.5, NFR-7.5	TC2	User is missing skills	FR-7.3, FR-8.1	NFR-2.7, NFR-3.5
TC3	User leaves a text box blank	FR-7.1, FR-7.2, FR-7.3, FR-7.4, FR-8.1	NFR-2.7, NFR-3.5, NFR-7.5	TC3	User is missing job experience	FR-7.3, FR-8.1	NFR-2.7, NFR-3.5
TC4	User leaves a text box blank	FR-7.1, FR-7.2, FR-7.3, FR-7.4, FR-8.1	NFR-2.7, NFR-3.5, NFR-7.5	TC4	No skill or job experience	FR-7.3, FR-8.1	NFR-2.7, NFR-3.5
TC5	User leaves a text box blank	FR-7.1, FR-7.2, FR-7.3, FR-7.4, FR-8.1	NFR-2.7, NFR-3.5, NFR-7.5				
TC6	User leaves a text box blank	FR-7.1, FR-7.2, FR-7.3, FR-7.4, FR-8.1	NFR-2.7, NFR-3.5, NFR-7.5				
TC7	User does not fill out resume	FR-7.1, FR-7.2, FR-7.3, FR-7.4, FR-8.1	NFR-2.7, NFR-3.5, NFR-7.5				

Fig 7.11. Functional and Non-Functional Traceability Matrices (UC7, UC8)



UC 9 TC ID	Scenario	Functional Req.	Non-Functional Req.
9.1	Successful removal from Favorites	FR-9.1, FR-9.2	NFR-3.5
9.2	Successful addition to Favorites	FR-9.1, FR-9.2	NFR-3.5
9.3	User quits mid selection	FR-9.1, FR-9.2	NFR-3.5
UC 10 TC ID	Scenario	Functional Req.	Non-Functional Req.
10.1	User quits mid selection	FR-10.1, FR-10.2, FR-10.3	NFR-3.5, NFR-2.8
10.2	Shared Successfully	FR-10.1, FR-10.2, FR-10.3	NFR-3.5, NFR-2.8
10.3	Shared Successfully	FR-10.1, FR-10.2, FR-10.3	NFR-3.5, NFR-2.8
10.4	Shared Successfully	FR-10.1, FR-10.2, FR-10.3	NFR-3.5, NFR-2.8

Fig 7.12. Functional and Non-Functional Traceability Matrices (UC9, UC10)

UC 12 TC ID	Scenario	Functional Req.	Non-Functional Req.
12.1	Successful rename		NFR-3.5, NFR-6.5, NFR-7.5
12.2	User quits when renaming		NFR-3.5, NFR-6.5, NFR-7.5
12.3	New name is empty		NFR-3.5, NFR-6.5, NFR-7.5
12.4	New name is too long		NFR-3.5, NFR-6.5, NFR-7.5
UC 13 TC ID	Scenario	Functional Req.	Non-Functional Req.
13.1	Successful Alignment	FR-13.1, FR-13.2	NFR-3.5, NFR-6.5, NFR-7.5
13.2	Successful Alignment	FR-13.1, FR-13.2	NFR-3.5, NFR-6.5, NFR-7.5
13.3	Successful Alignment	FR-13.1, FR-13.2	NFR-3.5, NFR-6.5, NFR-7.5
13.4	User quits when selecting alignment	FR-13.1, FR-13.2	NFR-3.5, NFR-6.5, NFR-7.5
UC 14 TC ID	Scenario	Functional Req.	Non-Functional Req.
14.1	Successful Order Swap	FR-14.1	NFR-3.5, NFR-6.5, NFR-7.5
14.2	Successful Order Swap	FR-14.1	NFR-3.5, NFR-6.5, NFR-7.5
14.3	Order Number Out of Range	FR-14.1	NFR-3.5, NFR-6.5, NFR-7.5
14.4	User quits mid input	FR-14.1	NFR-3.5, NFR-6.5, NFR-7.5
14.5	User quits mid click and drag	FR-14.1	NFR-3.5, NFR-6.5, NFR-7.5
UC 15 TC ID	Scenario	Functional Req.	Non-Functional Req.
15.1	Successful Section Addition	FR-15.1, FR-15.2	NFR-3.5, NFR-6.5, NFR-7.5
15.2	Successful Section Removal	FR-15.1, FR-15.2	NFR-3.5, NFR-6.5, NFR-7.5
15.3	Unsuccessful Section Removal	FR-15.1, FR-15.2	NFR-3.5, NFR-6.5, NFR-7.5
15.4	Unsuccessful Section Addition	FR-15.1, FR-15.2	NFR-3.5, NFR-6.5, NFR-7.5
15.5	User quits mid selection	FR-15.1, FR-15.2	NFR-3.5, NFR-6.5, NFR-7.5

UC 11 TC ID	Scenario	Functional Req.	Non-Functional Req.
11.1	Increase Font Size via click	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.2	Decrease Font Size via click	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.3	Increase Font via input	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.4	Decrease Font via Input	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.5	User quits when selecting font size	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.6	Font Size Max Reached (click)	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.7	Font Size Min Reached (click)	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.8	Font Size Out of Range (via input)	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.9	Font is Chosen	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.1	User quits when selecting font	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.11	Font Color is Chosen	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.12	User quits when selecting font color	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.13	Font Format is Selected	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.14	Font format is not selected	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5
11.15	Font is not changed	FR-11.1, FR 11.2, FR-11.3, FR-11.4, FR-11.5	NFR-3.5, NFR-6.5, NFR-7.5

Fig 7.13. Functional and Non-Functional Traceability Matrices (UC11, UC12, UC13, UC14, UC15)



8. SOFTWARE EVOLUTION

In order to implement our web application Resumaxing we execute the front end locally via HTML, CSS and JavaScript normally on a local development server. The backend is written in Python and is Flask-based, and it also operates in a different local port. The frontend requests the backend to read the information and reply to the user, during development the frontend uses HTTP requests sent to a localhost address, where the backend receives the data and responds to the user. Ollama is also incorporated to enable AI-based resume-generating and enhancements services. Supabase (AWS) manages our database and offers a scalable and cloud-based solution to store user data and generate resumes. Environment variables are used to handle sensitive information like API keys, database credentials, that are to be managed securely, to ensure that such data is not exposed in the codebase.

When our team is ready to deploy, we will make the necessary change to all API endpoints in the front end that refer to the live backend server and not localhost. Once deployed, the backend will be connected to the Supabase database so that it can have reliable and consistent data access. We will also test the entire system (frontend-backend communication, AI processing with Ollama, and database interactions) fully before it goes live to make sure that all processes work properly. We have been using Git and GitHub during development to work with each other and manage versions. This helps team members to work on different branches, monitor changes, check on pull requests, and address conflicts effectively. To improve quality of the codes, share knowledge and keep everyone on track of the newest edition of the project, peer reviews are regular.



GLOSSARY OF TERMS

WORD	DEFINITION
Use Case	A use case is a structured description of a system's behavior as it responds to requests from external actors, aiming to achieve a specific goal.
Functional Requirement	Functional requirements (FRs) define the specific behaviors, features, and operations a system must perform to meet user needs.
Non-Functional Requirement	Non-Functional Requirement (NFR) defines the quality attributes, operational constraints, and performance standards of a system.
Architecture Design	Architecture Design is the high-level process of defining the fundamental structure, organization, and interaction of a system's components.
Logical View	Logical View (often called the Logical Architecture) describes the abstract, structural design of a system required to fulfill its functional requirements.
Physical View	Physical View (also known as the Deployment View) depicts how software components and processes are mapped onto the physical hardware and network infrastructure.
System Modeling	System Modeling is the process of creating abstract, graphical representations (models) of a software system to understand, analyze, and communicate its structure, behavior, and functionality.
Wire Frames	Wireframe is a low-fidelity, structural blueprint of a user interface (UI) that outlines the layout, content placement, and functional flow of a system. It serves as a visual bridge between abstract requirements and the final coded implementation, focusing on "what" a page contains rather than "how" it looks aesthetically.
Implementation	Implementation is the phase where the design and architectural blueprints are transformed into a functioning software system. It is the transition from "what to build" to actually "building it."
Software Testing	Software Testing is the systematic process of evaluating and verifying that a software application or system functions correctly and meets its specified requirements. It involves executing a system or component to identify defects, gaps, or errors before the product is released to users.
Software Evolution	Software Evolution is the ongoing process of developing, maintaining, and updating software to ensure it remains useful and reliable as user needs and environments change.
UML diagram	The UML (Unified Modeling Language) diagram is a standardized visual representation of a software system's design, architecture, and implementation. It provides a common "visual language" for developers, architects, and stakeholders to communicate how a system is structured and how its components interact.
API	API (Application Programming Interface) is a set of defined rules and protocols that allow different software applications to communicate and exchange data with each other.
User Interface	User Interface (UI) is the specific point of interaction between a human and a computer system. It encompasses everything the user sees, touches, and hears—from buttons and menus to layouts and voice commands—acting as the "front window" of the software.



Frontend	Frontend (or "client-side") refers to everything a user interacts with directly in a software application. It encompasses the User Interface (UI), the visual design, and the logic that runs in the user's browser or on their device.
Backend	Backend (or "server-side") is the portion of a software system that remains invisible to the user. It is responsible for the "under-the-hood" operations: storing data, processing business logic, and managing security.
Router	A router is a networking device that forwards data packets between different computer networks.
Firewall	Firewall is a security system that monitors and controls incoming and outgoing network traffic based on predetermined security rules. It serves as a digital "gatekeeper," establishing a barrier between a trusted internal network (like a company's private intranet) and untrusted external networks (like the public internet).
N-Tier Architecture	N-tier architecture (also known as multi-tier architecture) is a client-server architecture in which presentation, application processing, and data management functions are physically and logically separated into distinct layers or "tiers."
Database	Database is an organized, electronically stored collection of interrelated data designed for efficient storage, retrieval, and management. It serves as the persistent storage layer for an application, ensuring that information—such as user profiles, transaction history, or inventory—remains available even after the software is closed, or the system restarts.
Expanded use case diagram	A visual representation that broadens the scope of a standard use case diagram to include multiple systems and end-to-end business processes
Sequence diagram	A type of Unified Modeling Language (UML) interaction diagram that illustrates how objects or components in a system interact with each other over time. It provides a chronological visualization of message exchanges required to carry out a specific scenario or use case.
Design class diagram	A type of static structure diagram that describes the structure of a system by displaying its classes, their attributes, methods (or operations), and the relationships among objects.
Test case	A set of conditions, inputs, execution steps, and expected results used to verify whether a specific feature of a software application functions correctly.
Developmental Methodology	A structured framework used to plan, manage, and execute the process of creating an information system. It provides a standardized "playbook" that defines specific roles, responsibilities, activities, and deliverables to ensure a project is completed within time and budget while meeting quality standards.
Waterfall developmental method	A traditional, linear, and sequential approach to the Software Development Life Cycle (SDLC). It is characterized by a "cascading" flow where each project phase must be fully completed and documented before the next phase begins.
Agile developmental method	An iterative and incremental developmental methodology that focuses on the continuous delivery of working software through rapid, flexible cycles. Unlike traditional linear models like Waterfall, Agile encourages constant feedback, adaptive planning, and close collaboration between self-organizing, cross-functional teams and their customers.
Manual testing	The process of manually executing test cases without the use of automation tools is necessary. Testers interact with the software just as an end-user to identify bugs, defects, and usability issues.
Unit testing	A level of software testing where individuals, the smallest testable parts of an application—called units—are isolated and verified for correctness. A "unit" typically refers to a single function, method, or class.



Integration testing	A level of software testing where individual units or components are combined and tested as a group. The primary goal is to expose defects in the interfaces and the interaction between integrated components.
Software Evolution	Software evolution is the continuous process of developing, maintaining, and updating software systems after their initial release to meet changing user needs, environmental shifts, and security requirements. It ensures software remains useful by adapting to new technologies, fixing bugs, and improving performance. It covers maintenance, enhancement, and re-engineering, often guided by Lehman's Laws.
Deployment	Software deployment is the critical process of making a software system available and ready for use by its intended audience. It acts as a bridge between the development phase and actual real-world usage, involving technical steps to move code from a development or staging environment to a live production environment.
MVC	MVC (Model-View-Controller) is an architectural pattern or design model that separates an application into three distinct components—Model (data), View (UI), and Controller (logic)—to manage complexity, improve maintainability, and allow parallel development. While it is a pattern, MVC is commonly implemented as a framework (e.g., ASP.NET MVC, Laravel, Django) to streamline web development.